System Governance and Specification Architecture: Integrating the Philosophy of Explainability into the Policy of Truth

1. Doctrinal Synthesis: Harmonizing Explainability with the Policy of Truth

The operational imperative to achieve complete specification coverage across all executable assets, combined with a mandate to increase systemic explainability by exactly twenty-five percent, requires a fundamental reconciliation of two foundational documents. These documents are the canonical Policy of Truth (PoT.md), which serves as the supreme constitution for repository governance [1], and the external Philosophy for Technical Documentation, which dictates the pedagogical structure of technical explanations.[1] The integration of these doctrines establishes a new operational paradigm for the Stela repository, transforming static specifications into rigorous, first-principles logic maps that serve both human Operators and autonomous AI agents (Integrators and Contractors).

The Policy of Truth dictates that systemic action is governed by hard physical laws, absolute literalism, and linguistic precision, explicitly stating that the system prevents action that violates policy rather than merely requesting compliance.[1] Conversely, the documentation philosophy introduces five key pedagogical directives: the Anti-Jargon Protocol, First-Principles Architecture, Anecdotal Anchoring, Rigorous Playfulness, and the Integrity Filter.[1] At first glance, the rigid, machine-enforced mechanisms of the repository—which trigger immediate "STOP WORK" conditions upon detecting ambiguity or state drift [1]—may appear at odds with organic, human-centric concepts such as "Rigorous Playfulness." However, a deeper architectural and epistemological analysis reveals that these frameworks are entirely synergistic. The philosophy of explainability acts as the semantic bridge that translates the binary, deterministic enforcement of the system into comprehensible narratives, thereby preventing operator-induced drift before it triggers automated rejection.

1.1 The Anti-Jargon Protocol as an Extension of Linguistic Precision

The Anti-Jargon Protocol demands the eradication of high-level abstractions and prestige language in favor of the simplest possible vocabulary, maximizing intellectual accessibility without sacrificing technical accuracy.[1] Within the constitutional framework of the repository, Section 4.2 of the Policy of Truth enforces a "Behavioral Logic Standard" characterized by "Linguistic Precision," explicitly prohibiting contractions across any scope and demanding "Absolute literalism".[1]

Jargon, by its very nature, is a linguistic contraction; it is a non-literal abstraction that relies heavily on unstated, assumed context shared exclusively between domain insiders. In an ecosystem heavily reliant on autonomous AI Contractor parsing—facilitated through discovery entry points such as llms.txt, llms-core.txt, and llms-full.txt—assumed context introduces a critical failure vector.[1] When an AI encounters an undefined piece of domain-specific prestige language, it may hallucinate a definition, leading to a direct violation of the Generation Mandate.[1] By applying the Anti-Jargon Protocol, technical specifications are systematically stripped of ambiguous terminology. This eradication process reduces the cognitive load on human Operators and minimizes the risk of hallucinatory drift by Contractor AIs. A twenty-five percent increase in explainability is achieved in this vector not by inflating word counts, but by optimizing semantic density. This involves replacing single, opaque jargon terms with explicit, mechanistic descriptions of systemic behavior, perfectly aligning with the mandate to seek clarification for ambiguity before proceeding.[1]

1.2 First-Principles Architecture and System Axioms

The external documentation philosophy includes a strict requirement to eliminate "black boxes" and to trace logic back to fundamental operational or physical foundations.[1] This directive maps directly onto Section 1.2 (Axioms) of the Policy of Truth.[1] The constitution asserts that "everything the system proves is actually true" and defines Truth as a "primitive predicate governed by specific rules rather than a mere definition".[1]

Consequently, a specification sheet for a critical binary, such as ops/bin/certify, cannot merely list its command-line arguments and exit codes. To fulfill the First-Principles Architecture directive, the specification must explain the existential foundation of the tool. It must state that certify exists because the system requires immutable verification of the Dispatch Packet (DP) contract, rendering a deterministic RESULTS receipt that mathematically proves the absence of state drift.[1] Furthermore, it must explicitly connect the tool's behavior to the Generation Mandate, explaining that manual fabrication of RESULTS is prohibited because human-authored receipts bypass the cryptographic hashes and untracked file analyses performed by the automation.[1] Every specification across the repository must answer the "why" by tracing its specific functionality back to the foundational axioms of the system architecture.

1.3 Anecdotal Anchoring and the Relatability Mandate

Abstract procedures must be grounded in concrete, real-world scenarios or "war stories" to transform dry technical data into a narrative of discovery and troubleshooting.[1] The repository's constitution already anticipates this requirement in Section 4.2 under the "Relatability Mandate," which calls for "Universal accessibility via vernacular optimization and anecdotal grounding".[1]

In practice, applying Anecdotal Anchoring to the forthcoming specification sheets involves systematically extracting historical incident data from the State of Play (SoP.md) ledger.[1] The State of Play is defined as the history ledger and shipment record, specifically designed to

provide context on *why* things are the way they are.[1] For example, when drafting the specification for ops/bin/prune, the author must reference past incidents to illustrate the tool's guardrails. The specification should point to the events of February 19, 2026, documented as DP-OPS-0074, where a premature prune execution led to the loss of a RESULTS artifact before certification was completed.[1] By embedding this specific historical failure into the documentation, the specification explains precisely why the uncommitted RESULTS guard violation triggers a hard stop during pruning operations.[1] This method transforms the specification from a passive list of arbitrary rules into an active, historically justified defense mechanism, fulfilling both the external philosophy and the internal Relatability Mandate.

1.4 Rigorous Playfulness Within Immutable Enforcement Boundaries

The documentation philosophy encourages an active, inquisitive mindset, allowing readers to "poke at the system" to truly understand its boundaries through experimentation.[1] Given that the Integrator and Automation mandates require killing processes, halting workers, and rejecting pull requests at the first sign of a "Dirty State" or "Canon Drift" [1], playfulness must be architecturally isolated to prevent catastrophic systemic locking.

This tension is resolved through strict adherence to the Filing Doctrine outlined in Section 1.1 of the Policy of Truth.[1] The var/tmp/ directory is officially designated as the "Resume" space for ephemeral worker scratch and task-local workspaces.[1] Specifications must explicitly guide users to conduct their experimental "play" exclusively within these defined boundaries. Furthermore, the concept of Rigorous Playfulness must be channeled into the explicit documentation of non-destructive testing modes. Tools such as tools/lint/dp.sh and ops/bin/prune feature --test and --dry-run flags.[1] The new specification standard must highlight these flags as the primary mechanisms for inquisitiveness, instructing Operators on how to safely simulate failure states and observe enforcement triggers without violating the non-negotiable rule against uncommitted changes in the immutable trunk.[1]

1.5 The Integrity Filter and the Equilibrium of Truth

The Integrity Filter mandates ruthless honesty regarding what is unknown, demanding that "best guesses," system limitations, or temporary workarounds be explicitly labeled as such, prioritizing the raw truth of the mechanism over the appearance of professional polish.[1] This principle fortifies the constitutional requirement for "Equilibrium maintenance" and addresses the axiom that "deception or incoherence are inherently destabilizing and must be resolved".[1]

If a script possesses known edge cases that lack deterministic handling, or if a legacy tool operates on a deprecated assumption, the new specification must state this fact openly. Hiding systemic fragility behind sanitized documentation constitutes an operational deception, which directly violates the Policy of Truth.[1] For instance, the historical ledger notes that during DP-OPS-0074, a legitimate enforcement-model gap existed between no-argument receipt scanning and strict hash parity checks, which required explicit operator authorization to bypass

and repair.[1] The Integrity Filter dictates that such architectural vulnerabilities, until permanently resolved, must be prominently documented in the relevant tool's specification sheet, ensuring that future Operators and AI Contractors are fully aware of the precise operational boundaries and failure risks.

Documentation Philosophy Directive	**Corresponding Constitutional Axiom / Directive**	**Architectural Execution Strategy for Specifications**
Anti-Jargon Protocol	Linguistic Precision & Absolute Literalism (§4.2)	Eliminate assumed context and prestige language. Define all operations using explicit, mechanistic vocabulary without contractions.
First-Principles Architecture	System Truth & Axioms (§1.2)	Eradicate black boxes. Trace all executable behavior back to constitutional constraints and physical repository laws.
Anecdotal Anchoring	Relatability Mandate & SoP History Ledger (§4.2, §1.3)	Inject SoP.md historical incident data directly into tool specifications to provide concrete, real-world justification for rules.
Rigorous Playfulness	Filing Doctrine (var/tmp/) & Verification Tooling (§1.1)	Document --test and --dry-run vectors prominently. Encourage boundary testing strictly within ephemeral, non-tracked constraints.
Integrity Filter	Action/Deception Incoherence & Equilibrium	Explicitly document unhandled edge cases, legacy debt, and

	(§1.2, §4.2)	non-deterministic hazards. Prioritize raw operational truth over polished presentation.

2. Comprehensive Diagnostic Assessment of the Executable Asset Corpus

To fulfill the mandate that absolutely all binaries, helper scripts, and linting tools possess a corresponding specification sheet, an exhaustive diagnostic inventory of the repository's executable assets must be established. The system dump and manifest records demonstrate that the repository is a highly structured environment tightly partitioned by the Filing Doctrine.[1]

The repository contains three primary execution domains that fall under the specification mandate: the binary execution layer located in ops/bin/, the subsystem routing and library layer located in ops/lib/scripts/, and the immutable enforcement and linting layer located in tools/lint/ and tools/test/.[1] Based on the cross-referencing of the active file tree against the documentation registry, the current state of specification coverage demonstrates strong baseline compliance but ultimately fails the absolute totality requirement of the new operational mandate. A substantial refactoring and backfilling operation is necessary.

2.1 The Binary Execution Layer Assessment (ops/bin/)

The binary directory acts as the primary operator interface for system interaction, state capture, and workflow progression. According to the OPS.md manifest and the repository index, the ops/bin/ directory contains a multitude of critical executables, including certify, compile, context, draft, dump, factory, help, llms, manifest, map, meta, open, project, prune, scaffold, and template.[1]

A review of the canonical documentation surfaces reveals that specifications exist in docs/ops/specs/binaries/ for a majority of these tools, such as certify.md, compile.md, context.md, dump.md, help.md, llms.md, map.md, open.md, project.md, prune.md, and template.md.[1] However, recent architectural fissions have created documentation drift. Specifically, historical ledger entries for DP-OPS-0077 reveal that the template binary underwent a structured fission into factory and manifest, while DP-OPS-0078 details the splitting of the project binary into scaffold and meta.[1] Because these fissions occurred rapidly to resolve operational bottlenecks, the corresponding specification sheets for factory, manifest, scaffold, and meta were not authored or formally registered. Furthermore, the critical draft binary, responsible for generating the immutable Dispatch Packet structure [1], entirely lacks an isolated specification sheet. To achieve 100% coverage, these missing documents must be generated and subsequently subjected to the new explainability philosophy.

2.2 The Subsystem Routing and Library Layer Assessment (ops/lib/scripts/)

The library scripts directory provides reusable guardrails, logic wrappers, and subsystem orchestration for the primary binaries. The active files in this domain include agent.sh, common.sh, factory.sh, heuristics.sh, ledger.sh, project.sh, skill.sh, synthesize.sh, task.sh, and traverse.sh.[1]

Existing specification coverage in docs/ops/specs/scripts/ includes agent.md, heuristics.md, project.md, skill.md, synthesize.md, and task.md.[1] This leaves a notable deficit. The common.sh file, which houses foundational utilities, lacks a spec. The factory.sh and ledger.sh scripts, which manipulate definitions and historical records respectively, are undocumented at the specification level. Additionally, the traverse.sh script, which was recently extracted from the dump binary to serve as a standalone filesystem traversal engine during DP-OPS-0078 [1], requires a dedicated specification sheet. The absence of documentation for these deep-layer scripts creates black boxes that violate the First-Principles Architecture directive.[1]

2.3 The Immutable Enforcement and Linting Layer Assessment (tools/)

The tooling directory houses the absolute laws of the system in executable form. These linters and test scripts are the manifestation of the Integrator's mandate to stop work, reject pull requests, and kill processes.[1] The tools/lint/ directory contains agent.sh, context.sh, dp.sh, factory.sh, integrity.sh, leaf.sh, llms.sh, project.sh, results.sh, schema.sh, skill.sh, style.sh, task.sh, and truth.sh.[1] The tools/test/ directory contains agent.sh, and the root tools/ directory contains verify.sh.[1]

The current specification coverage for this domain is highly robust, with docs/ops/specs/tools/lint/ housing documents for agent.md, context.md, dp.md, factory.md, integrity.md, llms.md, project.md, results.md, schema.md, style.md, task.md, and truth.md.[1] Specifications also exist for the verification and test scripts. However, the leaf.sh linter, introduced during the distributed leaf wiring completion in DP-OPS-0079 [1], and the skill.sh linter, lack distinct specification files. Because these linters act as hard gates during the continuous integration process [1], any opacity regarding their operational parameters causes severe friction for Operators attempting to resolve pipeline failures. Applying the Anti-Jargon Protocol and the Integrity Filter to these enforcement tools is critical for reducing mean time to recovery during failed dispatches.

2.4 Quantifying the Twenty-Five Percent Explainability Coefficient

The operator query demands a precise and measurable outcome: "explainability needs to be increased by 25% (explainability does not necessarily always mean/or equal an increase in words)" [User Query]. In a deterministic, policy-bound system, explainability cannot be

measured by narrative bloat. Instead, explainability (E) is defined as the inverse of the cognitive load required to successfully predict a tool's behavior and recover from its failure states.

To achieve a verifiable twenty-five percent increase in the explainability coefficient without relying on arbitrary word-count inflation, the text within every existing and newly created specification must be systematically restructured according to four rigorous semantic metrics:

1. **Elimination of Ambiguity Tokens**: The presence of generalized verbs such as "handles," "processes," "manages," or "updates" represents a failure of linguistic precision. These tokens must be targeted and replaced by explicit, mechanistic verbs. For instance, rather than stating that a tool "processes the manifest," the revised specification must state that it "parses YAML frontmatter, validates metadata-declared include references, and expands in-body include directives recursively".[1] This shift directly satisfies the Anti-Jargon Protocol.[1]
2. **Causal Dependency Mapping**: Explainability requires tracking cause and effect. Every command-line flag, environment variable, or required file path documented in a specification must explicitly state both its upstream precondition and its downstream systemic effect. If a parameter is missing, the specification must document the exact error code and failure sequence that will result.
3. **Integration of the "Why" Matrix**: To fulfill the First-Principles Architecture [1], every specification will receive a dedicated section that mathematically or logically justifies the existence of the tool against the PoT.md physical laws. If a tool cannot be justified by referencing the constitution, its existence is anomalous.
4. **Anecdotal Density Ratio**: The system will measure the number of historical incident citations drawn from SoP.md or PoW.md [1] embedded within the specifications. Increasing the presence of these "war stories" guarantees that abstract constraints are grounded in observable system history, fulfilling the Anecdotal Anchoring requirement.[1]

By systematically enforcing these four metrics across the entire corpus of specifications, the density of actionable information per paragraph will increase significantly, easily exceeding the requested twenty-five percent improvement in functional explainability without relying on superfluous prose.

3. Strategic Options Menu for Implementation

Achieving complete specification coverage while simultaneously applying a profound, metric-driven qualitative shift in documentation philosophy requires a highly controlled execution strategy. Because the Policy of Truth forbids "Dirty States," out-of-band modifications to the main branch, and mandates that all changes occur through the "Immutable DP Workflow" via small, reviewable work/* branches [1], the implementation must be meticulously serialized.

The strategy focuses on normalizing the underlying template engine for specifications, automatically enforcing the philosophical directives via continuous integration gates, and deploying autonomous and human resources to backfill and refactor the documentation corpus. Three distinct strategic pathways are available to execute this mandate. Each path strictly respects the non-negotiables of PoT.md, but they vary significantly in operational velocity, architectural risk, and resource utilization.

3.1 Strategic Option Alpha: The Staggered Domain Refactoring Protocol

Under this methodology, the integration of the explainability philosophy and the backfilling of missing specifications is treated as a series of standard, iterative feature developments. The Integrator generates a sequence of discrete Dispatch Packets (e.g., DP-OPS-0082 through DP-OPS-0086). Each DP is narrowly scoped to a single execution domain—for instance, the first DP solely targets the ops/bin/ directory, the second targets tools/lint/, and so forth.

An AI Contractor is deployed within each DP to manually analyze the source code of the targeted scripts, cross-reference the external documentation philosophy, and meticulously rewrite existing specifications or draft new ones for the missing assets. The Contractor must manually ensure that First-Principles thinking and Anecdotal Anchoring are woven into the text.

- **Pros**:
  - This approach is highly stable and architecturally safe. It maintains perfect equilibrium with the SoP.md ledger updates, as each domain overhaul is logged chronologically.[1]
  - The small, reviewable changesets perfectly align with Section 5.2 (Standard Workflow) rules, making human oversight and PR gating trivial.[1]
  - If a Contractor AI hallucinates a specification detail or violates the Anti-Jargon Protocol, the blast radius is tightly confined to a single PR that is easily caught and corrected during visual review.
- **Cons**:
  - Operationally time-intensive. Generating, executing, verifying, and closing five separate Dispatch Packets requires high operator overhead and slows system momentum.
  - Relies heavily on the interpretive capacity of the Contractor to maintain the exact philosophical tone across different DP batches, creating a risk of subtle tonal inconsistencies over time.
- **Risk Profile**: Low. The system's existing CI gates, specifically tools/lint/truth.sh and tools/lint/style.sh, will provide baseline defense against catastrophic canon drift, even if the process is slow.

3.2 Strategic Option Beta: The Immutable Structural Schema Overhaul

This option rejects relying on interpretive human or AI drafting and instead leverages the system's inherent deterministic automation. The primary action is the creation of a rigid,

machine-enforced template schema. The Stela repository already relies on a robust universal template engine (ops/bin/template) with strict slot validation to generate Dispatch Packets and Definition surfaces.[1] Option Beta extends this architecture to the specifications themselves.

A single Dispatch Packet is issued to create ops/src/surfaces/spec.md.tpl. This template hardcodes the five philosophical tenets as mandatory YAML frontmatter and injection slots (e.g., {{FIRST_PRINCIPLES_RATIONALE}}, {{MECHANICS_AND_SEQUENCING}}, {{ANECDOTAL_ANCHOR}}, {{INTEGRITY_FILTER_WARNINGS}}). Subsequently, tools/lint/style.sh is updated to forcefully reject any specification file that does not perfectly match this schema. Finally, a shell script is executed to forcefully parse all existing .md specs and wrap them in the new template structure, inserting `` flags into slots where the philosophical data does not yet exist, thereby triggering the Provisional Marking protocol outlined in PoT Section 4.5.[1]

- **Pros**:
  - Guarantees 100% adherence to the new philosophy through unyielding mechanical enforcement. It becomes physically impossible to merge a specification that lacks First-Principles justification or an Anecdotal Anchor.
  - Directly satisfies the "Generation Mandate" of PoT.md Section 4.2, which asserts that outputs must be generated rather than pre-filled or assumed.[1]
  - Centralizes the philosophical definition, allowing future adjustments to propagate automatically across all documents.
- **Cons**:
  - Will intentionally break the main build locally and flag massive linting failures until all `` slots are painstakingly resolved by the Operator and Contractor across the entire repository.
  - Forces a massive, repository-wide "Dirty State" during execution. This must be carefully managed within the var/tmp/ boundaries to prevent worker halts.[1]
- **Risk Profile**: Moderate. Changing the global template schema requires careful manipulation of ops/bin/template to ensure old specifications are properly migrated and that historical documentation data is not permanently lost during the wrapping process.

3.3 Strategic Option Gamma: The Synthesized Contextual Auto-Generation Pathway

This highly aggressive pathway leverages the existing "One Truth Context Engine" (ops/lib/scripts/synthesize.sh) to automate the entire mandate.[1] A new autonomous tool, tentatively named ops/bin/scaffold-spec, is rapidly developed. This tool programmatically reads the raw bash source code of every binary, script, and linter in the repository. It then feeds the raw code into an isolated LLM call, alongside a strictly formatted context payload containing the PoT.md constraints and the [1] philosophy document. The AI is instructed to bypass drafting and automatically generate the 100% covered, philosophy-compliant specification directory in a single, massive output stream.

- **Pros**:

    - Achieves immediate 100% coverage for all missing specifications (draft, factory, scaffold, leaf.sh, etc.) in a fraction of the time required by manual drafting.
    - Scales effortlessly into the future. As new enforcement tools are added to the repository, their specifications can be auto-generated to the exact philosophical standard without manual intervention.
    - Radically increases explainability by ensuring the documentation is a direct, algorithmic translation of the source code, virtually eliminating human interpretative error and drift.
- **Cons**:
    - Poses a severe risk of violating the AI Usage Policy defined in PoT.md Section 5.4, which mandates that all AI-proposed changes must be reviewed by a human operator.[1] Reviewing highly dense, auto-generated files for over thirty tools simultaneously is a cognitive burden that virtually guarantees oversight failures.
    - Automated extraction struggles fundamentally with "Anecdotal Anchoring." The bash source code itself does not contain the historical context of incidents (like DP-OPS-0074) required by the philosophy, meaning the AI will either leave these sections blank or hallucinate false history—a fatal system failure.[1]
- **Risk Profile**: High. Bulk auto-generation of canonical surfaces risks subtle hallucinations entering the core documentation, which would subsequently corrupt the context provided to future AI workers, initiating a cascading destabilization of the repository.

4. Proposed Implementation Strategy and DP-Sequencing

Recommendation: A Hybrid Execution of Option Beta (Structural Enforcement) and Option Alpha (Staggered Resolution).

To satisfy the dual mandate of securing 100% specification coverage and enforcing a rigid twenty-five percent quantitative increase in explainability via the designated philosophy, relying entirely on manual rewriting is too fragile, while bulk auto-generation carries unacceptable hallucinatory risks. A hybrid approach structurally forces compliance at the CI level while allowing for safe, verifiable content generation in isolated batches.

This plan orchestrates the transition through a series of highly controlled Dispatch Packets, adhering strictly to the Immutable DP Workflow outlined in Section 5.2.2 of the Policy of Truth.[1]

4.1 Step 1: Foundational Overhaul and Schema Locking (DP-OPS-0082)

The objective of the initial Dispatch Packet is purely structural. It establishes the physical laws required to enforce the new documentation philosophy.

1. **Initialization**: The Operator invokes ./ops/bin/open to establish the freshness state,

generating the required OPEN and OPEN-PORCELAIN artifacts. The Operator then generates the Dispatch Packet skeleton using ./ops/bin/draft --id=DP-OPS-0082.[1]

2. **Template Generation**: The Contractor creates a new canonical template at ops/src/surfaces/spec.md.tpl. This template utilizes the existing YAML frontmatter engine parsed by ops/bin/template.[1] It defines immutable required slots: {{FIRST_PRINCIPLES_RATIONALE}}, {{MECHANICS_AND_SEQUENCING}}, {{ANECDOTAL_ANCHOR}}, and {{INTEGRITY_FILTER_WARNINGS}}.
3. **Linter Hardening**: The Contractor modifies tools/lint/style.sh. The updated linter will parse all files in docs/ops/specs/ to ensure they conform to the new template. Furthermore, a "Jargon Blacklist" array is added to style.sh to mechanically enforce the Anti-Jargon Protocol [1], failing any build that uses prohibited prestige language.
4. **Closeout**: The DP is verified using ./tools/verify.sh and the updated linters. The ops/bin/certify binary is executed to generate the immutable RESULTS receipt.[1] The SoP.md is updated to record the structural integration of the philosophy.

4.2 Step 2: Asset Inventory and Boilerplate Stubs (DP-OPS-0083)

With the structural laws in place, the second DP identifies the deficits and stages the repository for the explainability refactor.

1. **Traversal and Mapping**: Using the ops/lib/scripts/traverse.sh engine [1], the Contractor conducts a comprehensive scan of ops/bin/, ops/lib/scripts/, tools/lint/, and tools/test/. This list is cross-referenced against the existing files in docs/ops/specs/.
2. **Stub Generation**: For every missing specification identified in Section 2 (e.g., factory, manifest, scaffold, meta, common.sh, leaf.sh), the Contractor utilizes the command ./ops/bin/template render spec to generate a boilerplate file.
3. **Provisional Marking**: Because the deep philosophical content cannot be written instantaneously, all generated slots are filled with `` tags, satisfying the Provisional Marking protocol in PoT.md Section 4.5 and preventing premature lint failures.[1]

4.3 Step 3: The Explainability Refactoring (DP-OPS-0084 to DP-OPS-0086)

The remaining work is batched by domain to maintain small, reviewable pull requests. DP-OPS-0084 targets Binaries, DP-OPS-0085 targets Scripts, and DP-OPS-0086 targets Tooling/Linters.

1. **Mechanistic Translation**: Within each DP, the Contractor analyzes the raw source of each tool and replaces the PROPOSED slots. To fulfill the Anti-Jargon Protocol and the 25% explainability metric, the Contractor translates the bash source into highly specific causal mappings.
2. **Applying First-Principles**: When documenting ops/bin/prune, the Contractor writes the {{FIRST_PRINCIPLES_RATIONALE}} slot by explaining that pruning is mathematically necessary to maintain token limits in LLM context windows while preserving audit history,

directly tying the script to system survival and the PoT.md.[1]

3. **Applying Anecdotal Anchoring**: The Contractor systematically reviews the SoP.md ledger. For the prune specification, the Contractor references DP-OPS-0074, noting: "As evidenced during DP-OPS-0074, premature execution of pruning leads to unrecoverable artifact loss; therefore, the uncommitted RESULTS guard is an absolute safety invariant".[1] This grounds the abstract rule in concrete, historical troubleshooting.
4. **Applying the Integrity Filter**: The Contractor documents any known edge cases. For instance, in the synthesize.sh specification, it must be explicitly noted that the script currently rejects any token containing a glob syntax, rendering dynamic recursive searches impossible by design.[1]

4.4 Step 4: Final Certification and Equilibrium Restoration

Following the completion of DP-OPS-0086, the repository will possess 100% specification coverage, with every document mathematically structured to deliver a 25% increase in actionable explainability.

For each individual DP throughout this process, the Operator must execute the strict closeout cycle. The ops/bin/certify binary is run to generate the deterministic RESULTS receipts, proving that the new specifications pass the aggressively upgraded style.sh and truth.sh linters without utilizing prohibited jargon or violating the PoT.md rules.[1] Finally, the SoP.md ledger is updated to record the permanent integration of the explainability philosophy into the core system mechanics, successfully altering the repository's baseline configuration while maintaining absolute systemic equilibrium.

Works cited

1. dump-full-main-6c63f583.txt